

Jersey Products Full-Stack Project Report

Github Repository

Table of Contents

1. Project Introduction
2. Early Development Process
3. Backend Setup and Controllers
4. Serializers and Data Formatting
5. Active Storage Integration
6. Frontend Clients (React and HTML)
7. Database Usage
8. Testing Strategy
9. Troubleshooting and Deployment Challenges
10. AI-Assisted Problem Solving
11. Reflection and Future Improvements

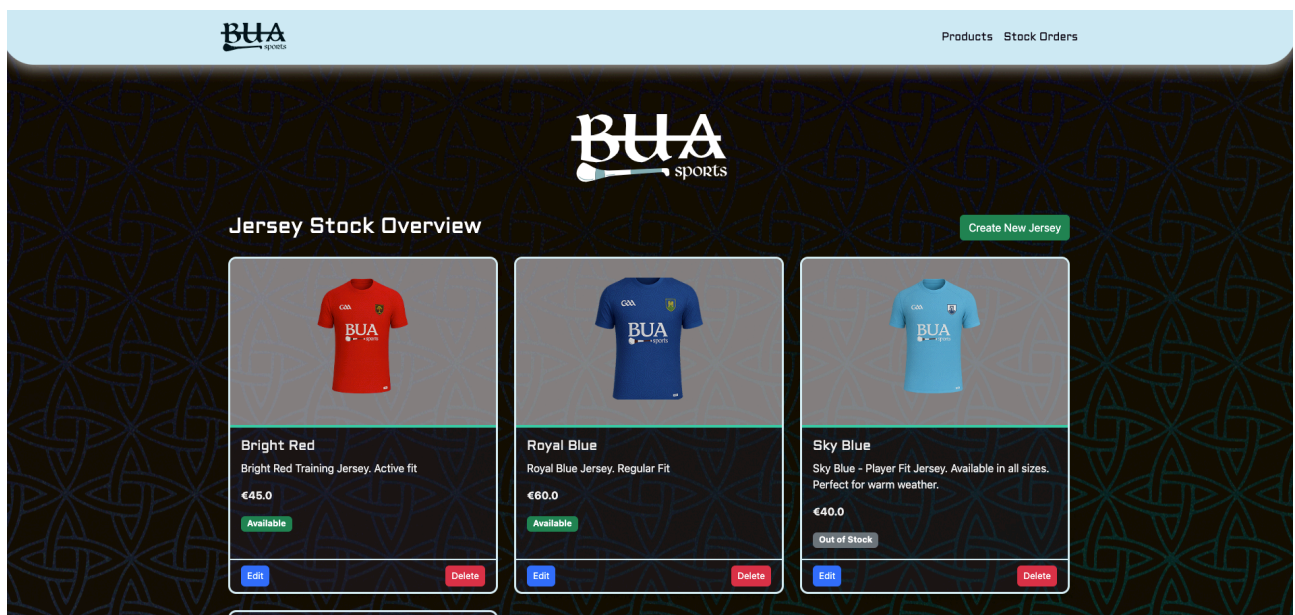
1. Project Introduction

This project set out to create an end-to-end full-stack application that would be able to manage products and stock orders for an admin system that supplies jerseys. The back end was written using Ruby on Rails, while the two front ends were implemented using different technologies; one was written in React, and the other was written using plain HTML as requested on the CA brief. The codebase is commented throughout. Both front ends communicate the Rails API. Active Storage is used in the app for image uploads.

2. Early Development Process

The project began by watching a suitable YouTube tutorial, which guided the creation of an airline review application. The tutorial allowed for a full conceptualisation of the project structure and gave a solid starting point. Ideas were adapted from the tutorial—airlines were replaced with products, airline reviews with stock orders—and there were some attempts to step away from the tutorial and make design decisions that better suited the look and feel of the project.

I set up Webpacker to handle the React frontend, paired it with Bootstrap, and began to slowly create a fully functioning app. My priorities were reliable CRUD functionality in both clients, and I avoided more advanced features that could create bugs. Ruby isn't my best language, but I managed to see this full-stack project through to a good working state that I am very happy with. I started out with a quick design art-board in Figma to organise some assets and envisage the way the app should look.



3. Backend Setup and Controllers

The backend was constructed with the standard Ruby on Rails API-only architecture. Two principal controllers were created: `ProductsController` and `StockOrdersController`. These controllers are nested within the `Api::V1` namespace.

The `ProductsController` managed actions like listing all products, showing one product only, creating a product, changing a product, and removing a product. The controller did this with the help of strong parameters that whitelisted fields like name, description, price, availability, and the image for Active Storage uploads.

Users could use the `StockOrdersController` to display all stock orders, make new stock orders that were connected to products, and delete stock orders. I added eager loading to the stock order index action to better the performance and avoid the N+1 query problem when fetching associated products. Eager loading, which is performed with `includes(:product)`, allows all needed data to be loaded at once.

Both controllers sent back JSON responses, using `FastJSONAPI` serializers to format the output neatly.

4. Serializers and Data Formatting

In this project, I employed two serializers: `ProductSerializer` and `StockOrderSerializer`, both derived from the `FastJSONAPI` gem.

The `ProductSerializer` defined attributes such as name, description, price, availability; and included a custom `image_url` attribute, for displaying uploaded product images. I modified the default image path generation, switching from `rails_storage_proxy_path` to `rails_blob_path`, to ensure the images could be correctly fetched in the frontend.

The attributes for quantity, sizes, comment, and `product_id` were handled by the `StockOrderSerializer`. I added a custom `product_name` attribute to those that the serializer was already handling. This allows the frontend to easily display the name of the jersey associated with each stock order without needing to perform additional API requests.

5. Active Storage Integration

One of the most challenging aspects of the project was integrating Active Storage to deal with image uploads. At first, I wasn't sure if I could implement Active Storage in a full-stack React and Rails environment given my skill level. I chose to proceed anyway, as doing so would greatly enhance the professional quality of the app.

To configure Active Storage, I generated the necessary migrations and executed the migrations with the command `rails db:migrate`. I updated the `Product` model to include the option `has_one_attached :image`. I configured the serializers and the controllers to allow image attachments.

During development, I encountered serious problems where the images uploaded would not display correctly in the React frontend. After many hours of troubleshooting, I asked ChatGPT to help figure out the issue, some of which involved following incorrect advice that had to be backtracked—I finally resolved the issue. I did this by adjusting the URL generation and by making sure that `ActiveStorage` routes were correctly mounted in `routes.rb`. The process of getting to the bottom of this matter taught me a great deal about file handling in Rails, as well as file handling in the integrated React-Rails environment. So the risk taken to resolve the image issue worth it, because it has significantly improved the visual user experience in my app.

6. Frontend Clients (React and HTML)

This project has two frontend clients.

The main user interface of the system is built with React and is served from Webpacker. This client provides the user with the ability to see products, upload new jerseys, edit products, and create stock orders. All React components make API calls to a Rails backend. The API is designed according to REST principles and uses JSON to encode data.

The form-based HTML client can be found at `/html-client.html`. It provides basic forms for creating products and stock orders. It does not rely on any JavaScript frameworks but has JS functionality most notably when displaying stock orders.

While deploying, I encountered the problem with the HTML client not communicating well with the backend. All attempts to create new products or stock orders through the HTML client were failing. The React frontend, however, was working perfectly. Seeing as time was becoming a precious commodity, I decided to just focus on the working React client and not worry about

debugging the HTML client to the point where it was in a deployable state. This is all documented in the video link.

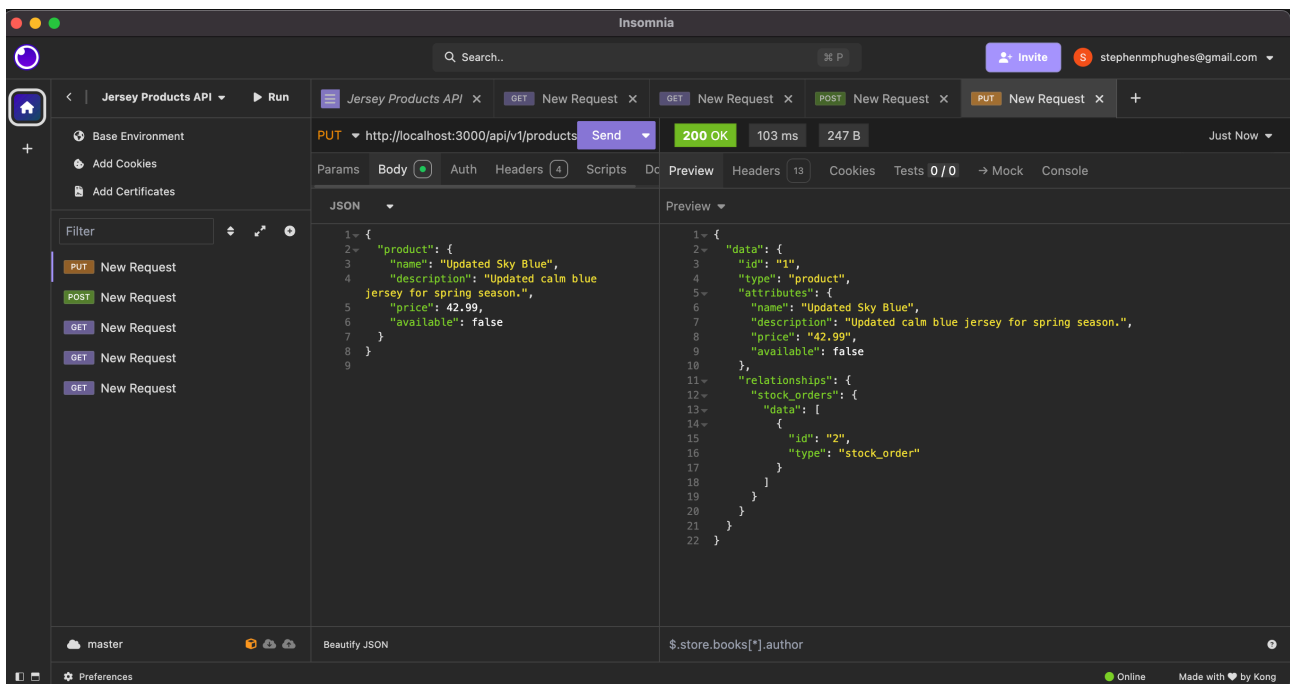
7. Database Usage

The project employs PostgreSQL for local development and for deployment. I set up the file config/database.yml to use PostgreSQL with standard development, test, and production settings. For the EC2 deployment, I created a PostgreSQL user and database through the terminal, then granted the necessary privileges.

I ensured that the database was properly set up to support Active Storage attachments and that every model was backed by a good database migration.

8. Testing

Initially I ran some API tests with insomnia to test the CRUD functionalities before any front end was developed.

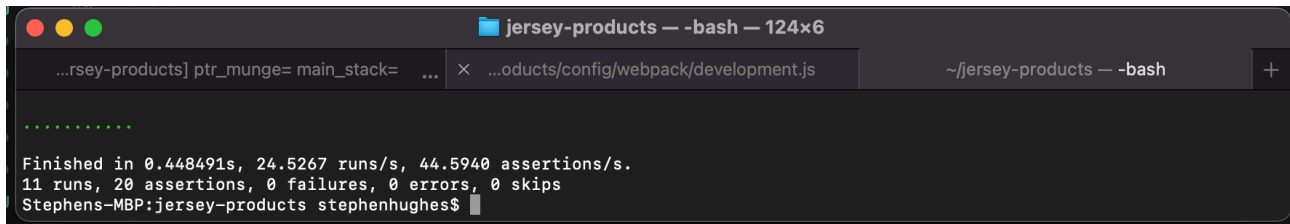


Further on, a comprehensive set of tests were carried out with Minitest, Rails' integrated testing framework. Front end was done with Jest.

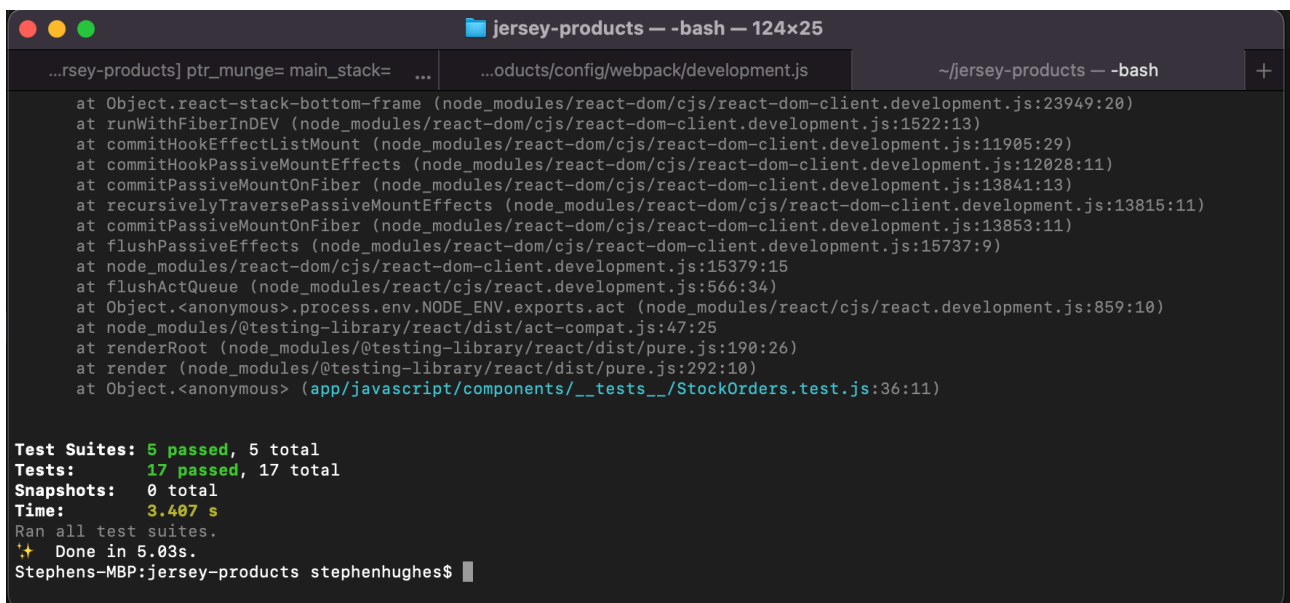
Model Tests: I wrote unit tests for both the Product and StockOrder models to ensure that the validation rules were enforced. A Product cannot be saved without a name, and a StockOrder cannot be saved without a quantity.

Integration Tests: An integration test file covered the creation, updating, and deletion of products and stock orders through the API. This ensures the entire data flow from request to database change works as expected.

Assessment of Challenges: A few small test failures were seen. For example, the HTML form test didn't pass at first because a <form> element doesn't inherently have a "form" role. I addressed this by modifying the tests to refer to the form via its ID as opposed to by ARIA role. There were a few issues that crept up but ultimately easy enough to solve.



```
jersey-products — -bash — 124x6
...rsey-products] ptr_munge= main_stack= ... X ...oducts/config/webpack/development.js ~/jersey-products — -bash +
.....
Finished in 0.448491s, 24.5267 runs/s, 44.5940 assertions/s.
11 runs, 20 assertions, 0 failures, 0 errors, 0 skips
Stephens-MBP:jersey-products stephenhughes$
```



```
jersey-products — -bash — 124x25
...rsey-products] ptr_munge= main_stack= ... ..oducts/config/webpack/development.js ~/jersey-products — -bash +
at Object.react-stack-bottom-frame (node_modules/react-dom/cjs/react-dom-client.development.js:23949:20)
at runWithFiberInDEV (node_modules/react-dom/cjs/react-dom-client.development.js:1522:13)
at commitHookEffectListMount (node_modules/react-dom/cjs/react-dom-client.development.js:11905:29)
at commitHookPassiveMountEffects (node_modules/react-dom/cjs/react-dom-client.development.js:12028:11)
at commitPassiveMountOnFiber (node_modules/react-dom/cjs/react-dom-client.development.js:13841:13)
at recursivelyTraversePassiveMountEffects (node_modules/react-dom/cjs/react-dom-client.development.js:13815:11)
at commitPassiveMountOnFiber (node_modules/react-dom/cjs/react-dom-client.development.js:13853:11)
at flushPassiveEffects (node_modules/react-dom/cjs/react-dom-client.development.js:15737:9)
at node_modules/react-dom/cjs/react-dom-client.development.js:15379:15
at flushActQueue (node_modules/react/cjs/react.development.js:566:34)
at Object.<anonymous>.process.env.NODE_ENV.exports.act (node_modules/react/cjs/react.development.js:859:10)
at node_modules/@testing-library/react/dist/act-compat.js:47:25
at renderRoot (node_modules/@testing-library/react/dist/pure.js:190:26)
at render (node_modules/@testing-library/react/dist/pure.js:292:10)
at Object.<anonymous> (app/javascript/components/__tests__/StockOrders.test.js:36:11)

Test Suites: 5 passed, 5 total
Tests: 17 passed, 17 total
Snapshots: 0 total
Time: 3.407 s
Ran all test suites.
🌟 Done in 5.03s.
Stephens-MBP:jersey-products stephenhughes$
```

9. Troubleshooting and Deployment Challenges

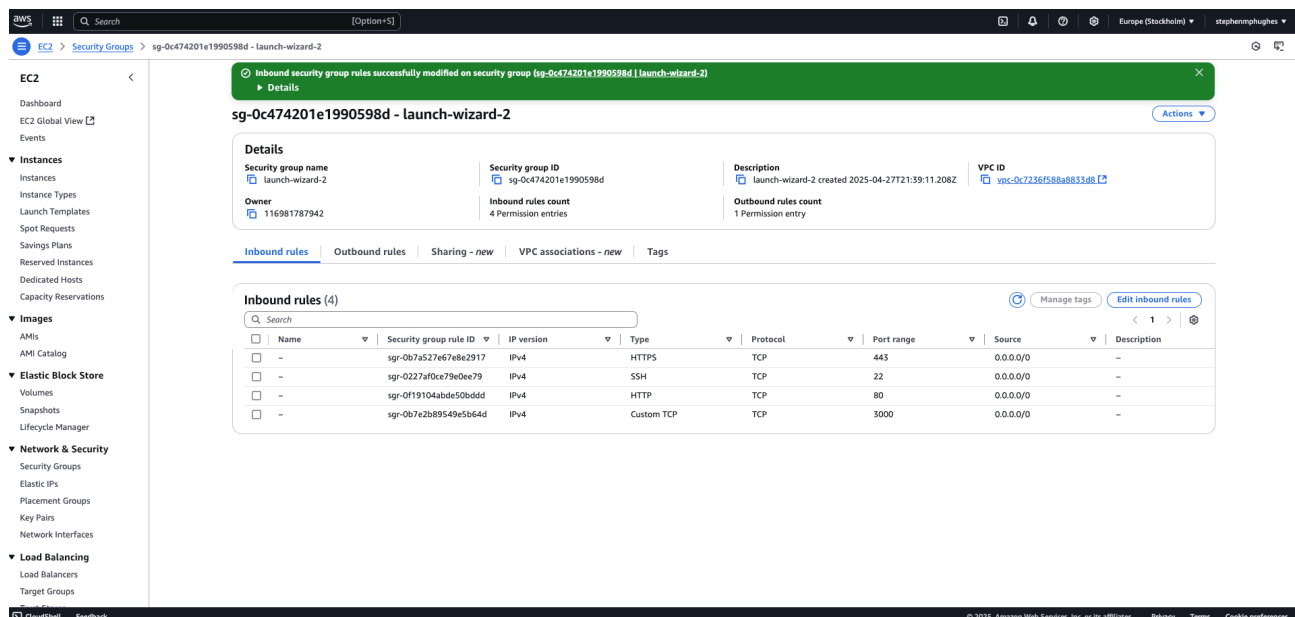
During the course of the project, I encountered several problems with deployments. At one time, there my EC2 instance had disk space available, but only after log files had been cleaned up and the system's cache cleared. After that, the installing of PostgreSQL, setting up of CORS, clearing of server PIDs, and general running of three different terminal tabs (one for backend, one for frontend, one for general commands) had to be done to get the app into a running state. And it was frequently necessary to do all of these again in order to re-deploy the app at all.

It was actually uncommon to not have to re-deploy at least one part of the app. When it wasn't necessary to do that, it was a relief as this is an area also where I lack.

Active Storage had some bigger issues. At first, the URLs that Rails generated for images did not work with React. After much trial and error, adjusting serializers, routes, and figuring out the proxying deal with Active Storage, I found a solid solution.

In the course of deployment, the HTML client encountered an error when attempting to interact with the backend. Since the HTML client was not functioning properly and the React client had no

known issues, the React client was deployed. A brief explanation is left in the video documenting the HTML client's issues.



10. AI-Assisted Problem Solving

Throughout this project, I used the AI tool ChatGPT to resolve a few technical problems I encountered. While the AI's recommendations sometimes sent me into dead ends from which I had to backtrack, they also helped me get through some frustrating loops.

Using AI to troubleshoot Active Storage was clearly a positive experience. I learned a lot from it, and I can now implement this feature more confidently in future applications. When you're serving files through Active Storage, there are two different kinds of paths that you can use: proxy paths and blob paths. I had been using blob paths, which are fairly easy to construct, but I had a problem with them.

Although I had assistance from AI at critical junctures, I ultimately required a good comprehension of the conventions of Rails and the integration of the frontend to completely resolve the issues.

11. Reflection and Future Improvements

Given a similar CRUD project in the future, I would set up filtered jersey dropdowns so that only available jerseys could be selected when a stock order was being created. I would also like to clean up the HTML client's communication layer between it and the server, and to properly debug the backend connection issues that have been causing trouble on my instance.

If I would have taken a more test-driven development approach from the start, the speed and reliability of the project overall would have been improved.

Integrating Active Storage was a risky decision, but it turned out to be highly rewarding, giving the application a feel that's much more professional.

Deployment Links:

React Frontend: <http://13.49.75.223:3000/>

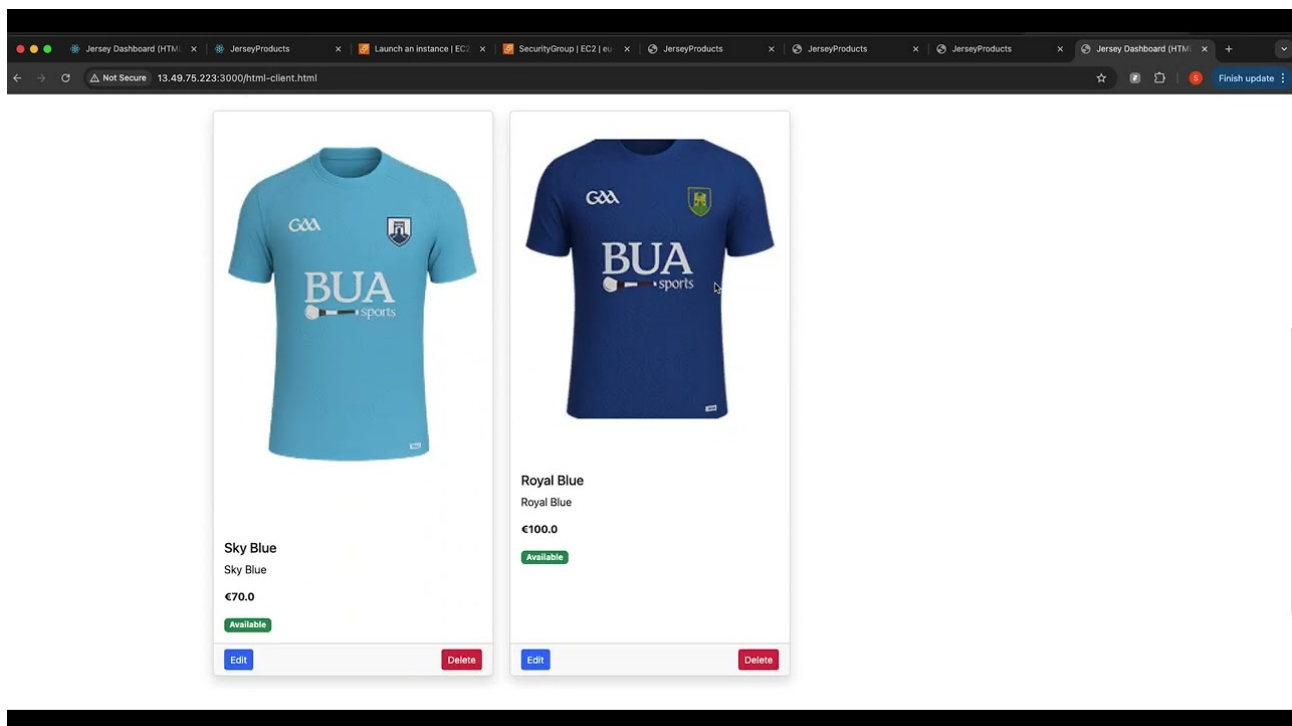
HTML Frontend: <http://13.49.75.223:3000/html-client.html>

APP on EC2 Video Link:

https://www.youtube.com/watch?v=juc7_MPxYgo

GitHub Repository: <https://github.com/stephenmphughes/Jersey-Products.git>

Video Embed With Visible Deyplyment URLs:



References:

- Ruby on Rails. (n.d.) *Action Controller Overview*. Available at: https://guides.rubyonrails.org/action_controller_overview.html
- Ruby on Rails. (n.d.) *Creating an API Application*. Available at: https://guides.rubyonrails.org/api_app.html
- Ruby on Rails Edge Guides. (n.d.) *Strong Parameters*. Available at: https://edgeguides.rubyonrails.org/action_controller_overview.html#strong-parameters
- Nopio. (n.d.) *Upload Files with Rails Active Storage*. Available at: <https://www.nopio.com/blog/upload-files-with-rails-active-storage/>
- Medium (@rail_to_rescue). (n.d.) *Eager Loading in a Ruby on Rails*. Available at: https://medium.com/@rail_to_rescue/eager-loading-in-a-ruby-on-rails-3c6aebba49cf#:~:text=Eager%20loading%20is%20a%20way,N%2B1%20query%20problem.%E2%80%9D
- Hands on React. (n.d.) *Testing Forms in React*. Available at: <https://handsonreact.com/docs/labs/js/T6-TestingForms>
- GeeksforGeeks. (n.d.) *How to Test React Components Using Jest*. Available at: <https://www.geeksforgeeks.org/how-to-test-react-components-using-jest/>

- Testing Library. (n.d.) *DOM Testing Library Examples*. Available at: <https://testing-library.com/docs/example-intro>
- Testing Library. (n.d.) *Screen API Tests*. Available at: https://github.com/testing-library/dom-testing-library/blob/main/src/_tests_/screen.js
- Mozilla Developer Network (MDN). (n.d.) *Promise.resolve()*. Available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/resolve
- Bootstrap. (n.d.) *Cards*. Available at: <https://getbootstrap.com/docs/4.0/components/card/>
- Mozilla Developer Network (MDN). (n.d.) *Template Literals*. Available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Template_literals
- DevMoran. (n.d.) *Handling CRUD Operations with React Hooks and Fetch API*. Available at: <https://devmoran.hashnode.dev/handling-crud-operations-with-react-hooks-and-fetch-api-a-hands-on-lab>
- DigitalOcean. (n.d.) *React CRUD Using Context and Hooks*. Available at: <https://www.digitalocean.com/community/tutorials/react-crud-context-hooks>
- Bootstrap. (n.d.) *Grid Layouts*. Available at: <https://getbootstrap.com/docs/5.3/layout/grid/>
- HackerNoon. (n.d.) *How to Upload an Image from React to the Rails API Using React Hook Form and Context API*. Available at: <https://hackernoon.com/how-to-upload-an-image-from-react-to-the-rails-api-using-react-hook-form-and-context-api>
- Mozilla Developer Network (MDN). (n.d.) *Using Fetch API*. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- Mozilla Developer Network (MDN). (n.d.) *Using FormData Objects*. Available at: https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest_API/Using_FormData_Objects
- Mozilla Developer Network (MDN). (n.d.) *Supplying Request Options with Fetch API*. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch#supplying_request_options
- GitHub (ankane). (n.d.) *Blazer Queries Test*. Available at: https://github.com/ankane/blazer/blob/master/test/queries_test.rb
- Ruby on Rails. (n.d.) *Integration Testing*. Available at: <https://guides.rubyonrails.org/testing.html#integration-testing>
- Ruby on Rails. (n.d.) *Test Helpers*. Available at: https://guides.rubyonrails.org/testing.html?utm_source=chatgpt.com#test-helpers